Claude Code 项目学习拆解

基于 `可借鉴部分.md` + 还原源码整理的系统学习型 PPT

目标：帮助你系统理解这个项目并迁移方法论

生成时间：2026-04-01

素材来源

1. `可借鉴部分.md` 的模式总结
2. `src/` 中关键实现文件
3. 仓库结构与恢复版工程特征

推荐使用方式

先通读，再按附录中的文件路径顺序去读源码。PPT 负责搭骨架，源码负责补细节。

交付物

本目录包含 deck 源码、可编辑 PPTX、渲染图与检验脚本输出，便于继续修改。

这套 PPT 怎么看

先建立认知地图，再回到代码细读，最后抽取可迁移的方法论。

- 项目全景
- 架构主线
- 关键模式
- MCP 与 Compact
- 安全与观测
- 迁移建议

适合对象

要详细学习这个项目的工程师、架构师、Agent/CLI 产品开发者。

阅读策略

每页只抓一个核心问题：它解决什么、为什么这么做、代价是什么。

最终产出

你应该能复述出 Claude Code 的启动链路、扩展点、压缩策略和安全边界。

WHY THIS REPO

为什么这个项目值得细学

它不是单点技巧集合，而是一个把 CLI 、 TUI 、 Agent 、 MCP 、压缩、权限、安全与观测串起来的完整系统。

完整产品面

从启动入口、命令系统到 TUI 、远程桥接、插件、技能、代理协作，都能在一个仓库里看到。

工程现实感

大量代码在处理启动性能、权限控制、断线重连、MCP 协议差异、上下文溢出等真实问题。

可迁移性高

很多模式可以直接迁移到你的 CLI 、 IDE 插件、 AI Agent 或平台工程里。

- 学习重点不是“抄源码”，而是识别设计决策背后的边界条件。
- 恢复版源码也很有价值，因为它暴露了真实系统的兼容层与工程妥协。
- 适合用来训练“如何读大型 TS 工程”的方法。

PROJECT SNAPSHOT

项目快照

先抓住几个可量化事实，建立规模感。

1364 行

学习文档 `可借鉴部分 .md`

2000+ 文件

当前工作树 `rg --files` 输出量级

80+ 命令

`src/commands.ts` 中集中注册

多层系统

CLI + React Ink TUI + services + tools + MCP

恢复版源码

包含 shim 、 vendor 、兼容逻辑与恢复痕迹

TypeScript-first

ESM + React JSX + Bun 宏特性

源码锚点： 可借鉴部分 .md | package.json | src/commands.ts

REPO MAP

仓库地图

学习大型仓库时，先按“入口 / 核心 / 横切 / 扩展 / 兼容层”分区。

入口层

`src/bootstrap-entry.ts`
`src/dev-entry.ts`
`src/main.tsx`
`src/commands.ts`

核心层

`src/query.ts`
`src/Tool.ts`
`src/tools/`
`src/tasks.ts`

服务层

`src/services/api`
`src/services/mcp`
`src/services/compact`
`src/services/analytics`

交互层

`src/components`
`src/ink.ts`
`src/interactiveHelpers.tsx`
`src/state/`

扩展层

`plugins`
`skills`
`shims`
`vendor`

- 先读入口，搞清楚程序是怎么启动、怎么接命令、怎么进入主循环。
- 再读服务层，理解横切能力： MCP 、 API 、压缩、分析、权限。
- 最后读扩展层，理解插件、技能、桥接与恢复版兼容逻辑。

LAYERED ARCHITECTURE

分层架构

这张图是理解全局最重要的一页：从命令入口一路走到工具、服务和 TUI 。

CLI Entry	`bootstrap-entry.ts` / `dev-entry.ts` / `main.tsx`
Command Layer	`commands.ts` 统一注册 + 条件加载
Core Processing	`query.ts` / `Tool.ts` / `tools.ts` / task loop
Bridge / Remote	bridge 、 remote session 、 transport 兼容层
Service Layer	analytics / mcp / compact / policy / oauth
React Ink TUI	Ink 渲染、组件、状态、交互辅助

源码锚点：可借鉴部分 .md 第 1 节

STARTUP FLOW

启动流程

启动链路的核心问题：怎样尽量早处理快速路径，怎样把重活并行化，怎样尽快进入 REPL/TUI 。

STEP 1
1. bootstrap-entry
解析 argv ，处理 `--version` 、系统提示导出等快速路径。

STEP 2
2. cli.tsx / main.tsx
进入 commander 与主程序入口。

STEP 3
3. 预取阶段
MDM 、 keychain 、 policy 、 bootstrap 数据等尽量并行启动。

STEP 4
4. 初始化阶段
telemetry 、 config 、 plugins 、 skills 、 MCP 、 LSP 。

STEP 5
5. renderAndRun
创建 AppState ，进入 Ink TUI / REPL 主循环。

你要重点观察什么
哪里做了早返回？哪里做了懒加载？哪些副作用被显式提前触发？

为什么值得学
启动链路最容易失控。这个项目示范了怎样把复杂系统拆成“快速路径 + 并行预取 + 主流程”。

源码锚点
`src/bootstrap-entry.ts`
`src/main.tsx`
`entrypoints/init.js`

STARTUP OPTIMIZATION

启动性能优化

`src/main.tsx` 开头的注释本身就是一份性能设计说明。

并行触发关键副作用

src/main.tsx

```
profileCheckpoint('main_tsx_entry')
startMdmRawRead()
startKeychainPrefetch()

// 目标：在其余 import 期间并行完成昂贵 IO
```

- 先记 profiling 点，再做重模块 import，便于量化“哪里慢”。
- 把 keychain / MDM 这种外部 IO 提前发出去，和模块加载并行。
- 注释直接写出大致收益：这是很成熟的性能工程习惯。

可以迁移到你的项目里的原则

把启动分成“绝对必须同步做的事”和“可以尽早发起但延后消费的事”。然后给每一类加 profiler/trace，而不是只靠体感优化。

源码锚点：src/main.tsx

关键设计模式

从构建裁剪到状态管理，这一段决定了项目的长期可维护性。

- Feature Flag
- 命令注册
- 消息类型
- Store
- Generator
- 缓存

阅读顺序

先看模式解决什么问题，再看具体实现。

阅读重点

构建时 vs 运行时、接口 vs 实现、缓存 vs 实时计算。

输出目标

能说清楚每个模式的收益、前提和代价。

PATTERN 1

Feature Flag + Dead Code Elimination

这是项目里最典型的“同一代码库服务多个产品形态”的实现。

构建时条件加载

src/commands.ts

```
const bridge = feature('BRIDGE_MODE')
  ? require('./commands/bridge/index.js').default
  : null

const voiceCommand = feature('VOICE_MODE')
  ? require('./commands/voice/index.js').default
  : null
```

- 用 `bun:bundle` 的 `feature()` 在构建时裁剪模块，而不是运行时 if/else 。
- 搭配 `require()` 可以避免静态 `import` 把整个模块都拉进来。
- 适合社区版 / 企业版 / 内部版 / 实验功能并存的仓库。

优点

零运行时分支开销；产物更小；发布矩阵灵活。

代价

构建系统更复杂；调试不同 feature 组合的成本上升。

判断标准

只有当“多产品形态”是长期事实时，这套机制才值得上。

源码锚点 : src/commands.ts | src/main.tsx

PATTERN 2

集中式命令注册表

`src/commands.ts` 把 80+ 命令拉到一个统一清单里，便于开关、过滤、元数据管理。

统一入口

src/commands.ts

```
const COMMANDS = memoize(() => [
  addDir,
  advisor,
  agents,
  branch,
  commit,
  ...(bridge ? [bridge] : []),
])

export const getCommands = () => COMMANDS()
```

收益 1

一眼看全命令面

收益 2

做条件注入和过滤

收益 3

配合 memoize 避免重复组装

- 命令对象统一携带 name / description / aliases / execute / prompt 等元数据。
- 远程模式只需要加一层 `filterCommandsForRemoteMode` 就能收窄能力面。
- 这是大型 CLI 从“脚本堆叠”升级成“命令平台”的关键一步。

PATTERN 2B

命令对象模型与远程过滤

统一注册只是第一步，更重要的是把命令能力显式类型化。

Command 基本字段

name / description / aliases / type / source / remoteIncompatible / execute / getPromptForCommand

三种命令形态

`prompt`：生成 prompt
`local`：本地执行
`local-jsx`：带交互 UI 的本地命令

远程模式策略

不是每个命令都能在 remote session 下成立，所以通过元数据统一过滤。

- 这种对象模型比“每个命令自己向 commander 注册”更容易做全局治理。
- 也是插件系统、技能命令、动态加载命令可以接入的前提。
- 如果你的 CLI 未来要支持本地 / 远程 / Web 容器，多半也要走这条路。

PATTERN 3

区分联合消息模型

这是 TypeScript 工程里非常值得借鉴的一种“类型驱动设计”。

Message 联合类型

src/types/message.ts

```
type UserMessage = { type: 'user', ... }
type AssistantMessage = { type: 'assistant', ... }
type SystemMessage = { type: 'system', ... }

type Message =
  | UserMessage
  | AssistantMessage
  | SystemMessage
```

- 用 `type` 作为 discriminator， switch 分支里自动收窄类型。
- 新增消息类型时，编译器会逼你补齐处理逻辑，减少漏改。
- 比深层继承更轻，尤其适合消息、事件、协议包这类数据结构。

学习重点

留意这个项目里哪些地方在用 union + exhaustive matching 。它们往往是系统边界最清楚、最不容易写乱的地方。

PATTERN 4

50 行可观察 Store

`src/state/store.ts` 是一个典型的“小而够用”的状态管理实现。

createStore

src/state/store.ts

```
export function createStore(initialState, onChange) {
  let state = initialState
  const listeners = new Set()
  return {
    getState: () => state,
    setState: updater => {
      const next = updater(state)
      if (Object.is(next, state)) return
      state = next
      onChange?.({ newState: next, oldState: prev })
      for (const listener of listeners) listener()
    },
```

为什么成立

需求明确，状态规模可控，不需要 Redux 那套复杂度。

关键细节

`Object.is` 恒等检查 + `unsubscribe` 闭包。

适用边界

团队要接受“简洁而非全功能”。

- 先问自己：项目真的需要外部状态库吗？还是一个几乎零心智负担的小 store 就够？
- 这个实现适合可控内聚的 TUI/ 桌面交互状态，不适合跨边界共享的复杂业务域。

PATTERN 5

Generator 与 Memoize

一个处理流式数据，一个避免重复 I/O；都属于“让系统更省”的手法。

Generator： 流式处理工具结果

典型位置： `src/query.ts`

- 用 `yield` 逐个吐出缺失工具结果，而不是先拼完整数组
- 适合大量消息、工具块、流式返回的场景
- 优点是惰性、可中断、可管道化

Memoize： 会话级上下文缓存

典型位置： `src/context.ts`

- git status / git log / user context / claude.md 扫描都做懒计算
- CLI 会话里这些信息通常“频繁访问、变化不快”
- 适合用内存换响应时间

共同点

都不是“炫技模式”，而是围绕成本控制：少分配内存、少做 I/O 、少重复计算。大型 Agent/CLI 项目普遍需要这种手感。

PATTERN 6

CLI 快速路径设计

`bootstrap-entry.ts` 的价值不在“跳转入口”，而在“尽早提前退出”。

STEP 1
--version
只打印版本，不拉起完整主程序。

STEP 2
--dump-system-prompt
只加载提示相关模块。

STEP 3
daemon / remote
走部分模块与专用分支。

STEP 4
interactive
才进入完整 cli.tsx / main.tsx 。

- 做 CLI 时，最常见的错误是“无论干什么都先把整个应用启动起来”。
- 快速路径不是优化细节，而是用户体验的大头，尤其是 `--help` / `--version` 这类高频操作。
- 如果你的程序还有 cron 、 daemon 、 worker 、 non-interactive 模式，更应该显式拆出来。

PATTERN 7

桥接层多态： Bridge v1 / v2

同一个调用接口，底下可以挂不同传输协议，这是协议演进时的常用手法。

统一接口

`ReplBridgeTransport`
– write(message)
– writeBatch(messages)
– close()

v1 实现

HybridTransport
WebSocket 双向通信
适合旧协议和兼容期

v2 实现

SSETransport + CCRClient
读写分离： SSE 读、 HTTP 写
适合新架构渐进迁移

- 调用方无需关心底层是 WS 、 SSE 还是 HTTP ， 只依赖最小接口。
- 迁移时可以 v1/v2 共存，降低“切协议即重写业务层”的风险。
- 这是接口隔离原则在通信协议上的一个典型落地。

PATTERN 8

统一重试与认证恢复

这个仓库不是只“重试请求”，而是在认真区分错误类型和调用场景。

OAuth 重试思想

文档中展示了 `withOAuthRetry()`：先调一次，401 时刷新 token，再用新 token 补一次。价值在于把认证恢复逻辑从每个 API 调用点里抽走。

`src/services/api/withRetry.ts` 的现实版

- 区分 529 / 429 / stale connection / auth error
- 根据 querySource 决定要不要重试
- 支持 unattended retry、fallback、fast mode cooldown

真正值得借鉴的点

不是“写个 while 重试”，而是把“可重试性判断”“重试次数”“调用来源”“降级策略”都模型化。

源码锚点：可借鉴部分 .md 2.4 | src/services/api/withRetry.ts

MCP 与上下文管理

Claude Code 的真正复杂度之一，来自它需要同时面对多协议、权限、配置和上下文预算。

- MCP transport
- normalization
- policy
- compact
- tool pipeline

为什么重要

MCP 是工具生态入口， Compact 是大上下文对话能否持续的关键。

怎么看

先看连接与配置，再看名字标准化，再看上下文压缩。

你要带走什么

一套“多协议 + 多预算 + 多边界”的系统设计思路。

MCP 多传输支持

`src/services/mcp/client.ts` 体现的是“连接管理”与“协议细节”解耦。

stdio
本地进程 stdin/stdout

sse
Server-Sent Events

http
streamable HTTP

websocket
双向实时通道

MCPClientManager 的职责
1. 根据 config 决定 transport
2. 建立 / 维护连接 map
3. 管理生命周期 start / close
4. 对上提供统一工具 / 资源能力

为什么这个结构靠谱
新增协议时主要改 transport factory，而不是散落修改调用方。适合跟随生态演进逐步扩展。

源码锚点：可借鉴部分 .md 第 3 节

MCP SAFETY

MCP 名称标准化与策略过滤

真正的系统不会只“连上就行”，它还要处理命名约束、政策边界和企业配置。

normalizeNameForMCP

src/services/mcp/normalization.ts

```
const CLAUDEAI_SERVER_PREFIX = 'claude.ai '

export function normalizeNameForMCP(name) {
  let normalized = name.replace(/[^a-zA-Z0-9_-]/g, '_')
  if (name.startsWith(CLAUDEAI_SERVER_PREFIX)) {
    normalized = normalized.replace(/_+/g, '_').replace(/^_|_$/g,
'')
  }
  return normalized
}
```

- 命名标准化不是小事，它直接影响 API 合法性、 tool name 拼接和跨系统兼容。
- `main.tsx` 里还能看到 policy limits 、 enterprise mcp config 、 server exclusion 等治理逻辑。
- 这说明 MCP 在这里不是单纯“插件协议”，而是受控能力面。

给你的提醒

只做 transport 不够，名字规范、权限限制、配置合并、组织策略同样必须设计。

源码锚点 : src/services/mcp/normalization.ts | src/main.tsx

COMPACT

Compact 服务全景

上下文压缩不是单一算法，而是一套“何时压、压什么、压完如何补”的流程。

触发面
token budget
message count
memory usage
manual / proactive

策略面
full
auto
micro
memory
reactive

执行面
分析消息
提炼摘要
插入边界
恢复关键上下文

补偿面
重注入文件
重注入技能
hook
session metadata

目标
保住可继续对话的最小知识面

- 高水平设计不在“压缩”本身，而在压缩前后对系统一致性的维护。
- Claude Code 的 compact 代码量很大，恰恰说明这个问题在真实产品里非常棘手。
- 如果你的 Agent 会长时间运行，这部分几乎绕不过去。

源码锚点：可借鉴部分 .md 第 4 节 | src/services/compact/

COMPACT INTERNALS

Compact 细节：真正麻烦的地方

`src/services/compact/compact.ts` 里有很多“只有做过大系统才会想到”的保护逻辑。

stripImagesFromMessages()

压缩摘要时剥掉 image/document，避免 compaction 请求自己再撞上 prompt-too-long。

stripReinjectedAttachments()

对会在压缩后重新注入的附件先剥掉，避免摘要里混入过期技能提示。

PTL / retry / restore

处理 prompt-too-long、streaming retry、post-compact file budget、skill budget。

核心启发

不要把压缩看成“把旧消息总结一下”这么简单。只要系统里存在图片、附件、技能注入、hook、计划文件、文件缓存，compact 就必然变成一个恢复流程。

源码锚点：src/services/compact/compact.ts

TOOL PIPELINE

工具执行主线

仓库里有大量工具与代理能力，理解“工具从哪里来、怎么被调度、如何反馈结果”很关键。

STEP 1
getTools()
汇总 builtin tools 、 MCP tools 、 synthetic tools 。

STEP 2
tool orchestration
决策工具可用性、权限、调用顺序。

STEP 3
execution / streaming
实际执行并逐步产生结果块。

STEP 4
message synthesis
把 tool_use / tool_result 回填为消息。

STEP 5
next round
进入下一轮 query / compact / UI 更新。

- 源码锚点可以先看 `src/Tool.ts` 、 `src/tools/` 、 `src/services/tools/` 。
- 理解工具主线后，再回头看 AgentTool 、 MCP 工具、 FileReadTool 等会容易很多。
- 大多数 Agent 系统最后都会回到这条“统一工具协议 + 调度 + 回填消息”的主线。

工程化与运行质量

一个成熟项目不会只关注功能，还会把性能、安全、观测、扩展和恢复性做成一等公民。

- performance
- security
- observability
- plugins
- agents

这一段怎么看

把它当作“生产级软件的收口部分”。

重点问题

系统如何自保？如何排错？如何扩展而不失控？

迁移价值

这部分比语法技巧更能拉开工程成熟度差距。

OPERABILITY

性能实践亮点

这不是“写完再优化”，而是在架构层面预埋性能思维。

- 快速路径：`--version` 这类命令不启动完整应用。
- 并行预取：keychain / MDM / bootstrap 数据尽量提前发起。
- 条件加载：feature flag + lazy require 降低冷启动开销。
- memoized context：git / markdown / memory 文件避免重复扫描。
- 按 querySource 调整重试：不要让后台任务把前台体验拖死。

真正值得学习的地方

1. 代码注释敢写出性能动机与大致收益
2. profileCheckpoint 这类设施进入主干
3. 性能优化分散在启动、重试、缓存、模块裁剪等多个层级

一句话总结

把“少做事”“晚做事”“并行做事”“只在必要时做事”四件事做到位。

SECURITY

安全与边界控制

虽然这是恢复版源码，但依然能看到明显的防护意识。

分析日志的类型约束

`analytics/index.ts` 明确限制 metadata 不要随手塞字符串，避免把代码 / 路径 /PII 打进通用日志。

MCP / policy / permission

主程序初始化时就会载入 policy limits 、managed settings 、 MCP 配置过滤。

代理与工具控制

工具执行、 remote mode 、 session ingress 、sandbox/worktree 等都在做能力边界划分。

- 这类系统的安全不是单点校验，而是“配置 + 权限 + transport + 审计”的组合防线。
- 你可以重点搜 `policyLimits` 、 `permission` 、 `sandbox` 、 `secureStorage` 、 `secretScanner` 这些关键词。
- 真正的经验是：能力越强的 Agent 工具链，越要在边界治理上花代码量。

源码锚点 : src/services/analytics/index.ts | src/main.tsx

OBSERVABILITY

可观测性与调试面

一个复杂 Agent/CLI 项目如果没有观测，问题会变成不可解释。

启动观测

profileCheckpoint / profileReport / startup profiler

事件观测

analytics sink / first party logs / datadog / statsig

诊断日志

debug / internalLogging / toolUseSummary / diagnostics

为什么这很关键

在多模式、多协议、多代理、多工具的系统里，“功能对不对”只是问题的一半；另一半是“出了问题你能不能定位到是哪个阶段、哪个配置、哪个 transport 、哪个预算导致的”。

源码锚点：src/services/analytics/* | src/utils/debug.js | src/utils/startupProfiler.js

EXTENSIBILITY

插件与技能系统

这个仓库的扩展不是附属品，而是产品能力的一部分。

插件系统

`src/services/plugins`
`src/utils/plugins`
`src/commands/plugin`

负责安装、缓存、校验、命令暴露与设置流转。

技能系统

`src/skills`
`bundledSkills`
`loadSkillsDir`
`skillSearch`

负责把本地 / 远程技能接入到模型可用能力面。

设计意义

命令、插件、技能都被建模成可发现、可加载、可过滤的对象，而不是零散脚本。

迁移启发

如果你的系统未来也要支持 marketplace 、插件生态、技能包或企业定制，这一套“统一对象模型 + 缓存 + 命令集成”非常值得拆开看。

AGENTS

Agent 与协作模式

从文件结构就能看出：Claude Code 不是单线程单 Agent 的玩具，而是在为多代理和协作模式留空间。

内置 agent 定义

`src/tools/AgentTool/built-in/*`
plan / explore / verify / guide 等角色化 agent

协作与 swarm

`src/coordinator/`
`src/utils/swarm/`
`src/utils/teammate.js`

状态与恢复

sessionStorage / reconnection / remote session / task state

- 这意味着很多设计决定都不能只按“单轮对话”来评估，而要按多线程、多 agent 、多 session 来评估。
- 如果你在做 agent 平台，这一段可以反过来帮助你定义最小可行抽象：agent definition 、 task state 、 message protocol 、 tool boundary 。

RESTORED TREE

恢复版源码的阅读姿势

这是恢复版，不是 pristine upstream 。这个事实会影响你对代码质量与边界的判断。

你会看到什么

shim 、 vendor 、本地包替身、兼容分支、懒加载修补、 ant-only 注释与 feature gate 。

怎么解读

区分“核心设计模式”与“恢复 / 兼容妥协”，不要把所有 workaround 当最佳实践。

为什么仍然有价值

真实系统往往就长这样：核心设计 + 历史包袱 + 多版本兼容。

- 看架构：学模式；看 workaround ：学约束与演化成本。
- 恢复版最有价值的地方，恰恰是它暴露了“工程现实”而不是演示版洁癖。
- 但在迁移时要主动清洗：只复制你能解释清楚的设计。

把经验迁移到你的项目

最后一段不再讲“它做了什么”，而是讲“你应该怎么借”。

- 先借什么
- 不要盲抄什么
- 阅读顺序
- 附录索引

目标

把 37 页内容收敛成可执行的行动项。

方法

先选模式，再找适用边界，再做本地化改写。

原则

宁可少借，别把复杂度整体搬进来。

MIGRATION

优先借鉴的 8 件事

如果你没有时间通盘复制，先借这些最稳、收益最高的做法。

- 快速路径入口：`--version` / worker / daemon 分流
- 集中命令注册表：让 CLI 命令成为可治理对象
- Feature flag + lazy require：只在确实多产品时使用
- 区分联合消息模型：消息 / 事件系统更稳
- 小型 observable store：替代过度复杂的状态库
- 会话级 memoize：缓存 git、md、配置等昂贵读取
- 统一重试层：按错误类型与调用来源决策
- 观测优先：profile、analytics、debug 设施进主干

TRADE-OFFS

不要盲抄的部分

很多设计只有在这个项目的复杂度前提下才成立。

重型 feature matrix

如果你只有单一产品形态，就不要为了“未来可能需要”先上满 feature flag 。

大而全 compact

如果对话长度不大、附件类型不多，就别过早复制这一整套恢复流程。

过多 transport

先支持最小必要协议；协议数量本身就是维护成本。

- 判断标准很简单：你能不能说清楚它解决的痛点已经真实存在？
- 如果答案是否定的，就先保留简单实现，把演化钩子留好即可。
- 架构成熟不等于复杂度越高越好，而是复杂度和约束相匹配。

STUDY PLAN

推荐源码阅读顺序

照这个顺序读，你会更快建立因果关系。

STEP 1

Step 1

入口：`src/bootstrap-entry.ts` → `src/main.tsx` → `src/commands.ts`

STEP 2

Step 2

核心循环：`src/query.ts` / `src/Tool.ts` / `src/tools/`

STEP 3

Step 3

服务横切：`src/services/api` / `mcp` / `compact` / `analytics`

STEP 4

Step 4

交互层：`src/ink.ts` / `components` / `state/store.ts`

STEP 5

Step 5

扩展层：`skills` / `plugins` / `AgentTool` / `coordinator`

经验

每读完一层，先回答“它为上一层提供了什么稳定接口”，再继续往下。

APPENDIX

附录：重点文件索引

这页可以当成你的源码导航清单。

核心入口与主干

- `src/bootstrap-entry.ts` 入口快速路径
- `src/main.tsx` 主启动链路与初始化
- `src/commands.ts` 命令注册总表
- `src/context.ts` memoized context
- `src/query.ts` 消息 / 工具主循环
- `src/Tool.ts` 工具模型
- `src/state/store.ts` 极简 store
- `src/services/api/withRetry.ts` 重试策略

服务与扩展

- `src/services/analytics/index.ts` sink/ 日志约束
- `src/services/mcp/normalization.ts` 名称规范化
- `src/services/mcp/client.ts` 多 transport
- `src/services/compact/compact.ts` 压缩主实现
- `src/tools/AgentTool/` agent 能力
- `src/utils/plugins/` 插件加载与缓存
- `可借鉴部分 .md` 这次 PPT 的讲义母本